

F. Multimodal Inference and generation

F.1 Introduction to Multimodal Representation & Alignment

F.2 Multimodal Fusion

Motivation

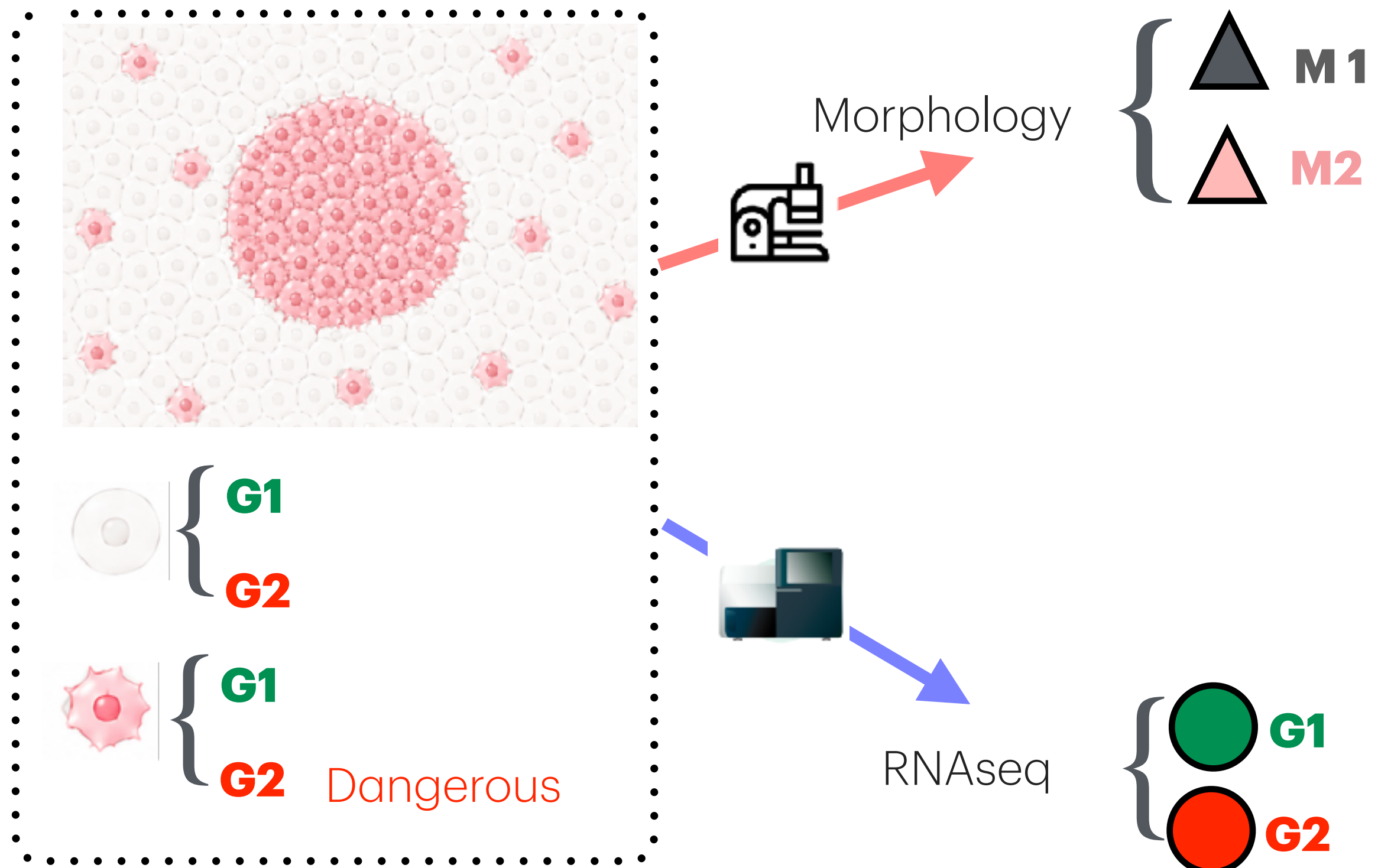
Use what each modality adds, fusion design space

F. 3 Cross-Modal learning

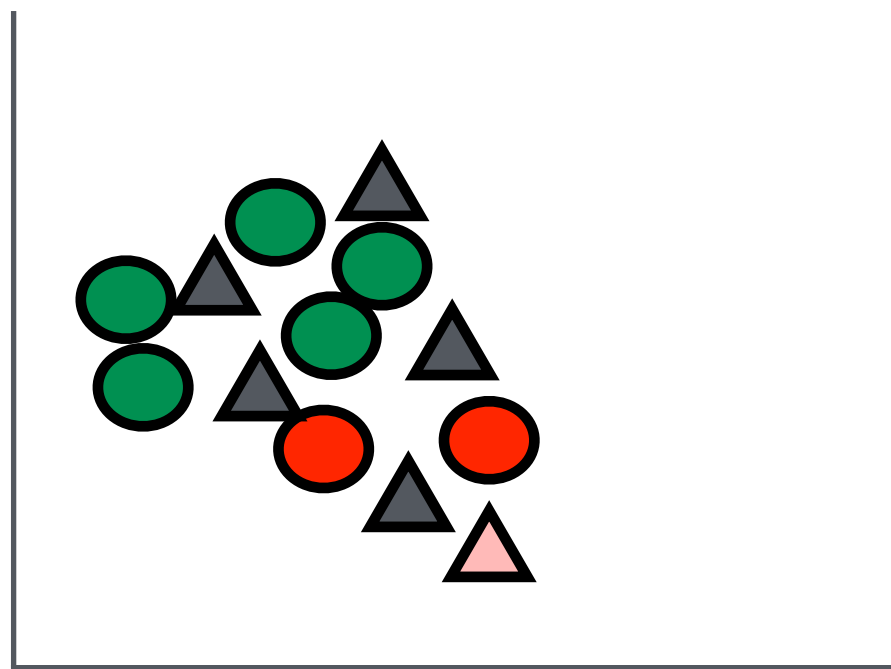


Tiam Heydari, PhD
Postdoctoral Fellow, UBC

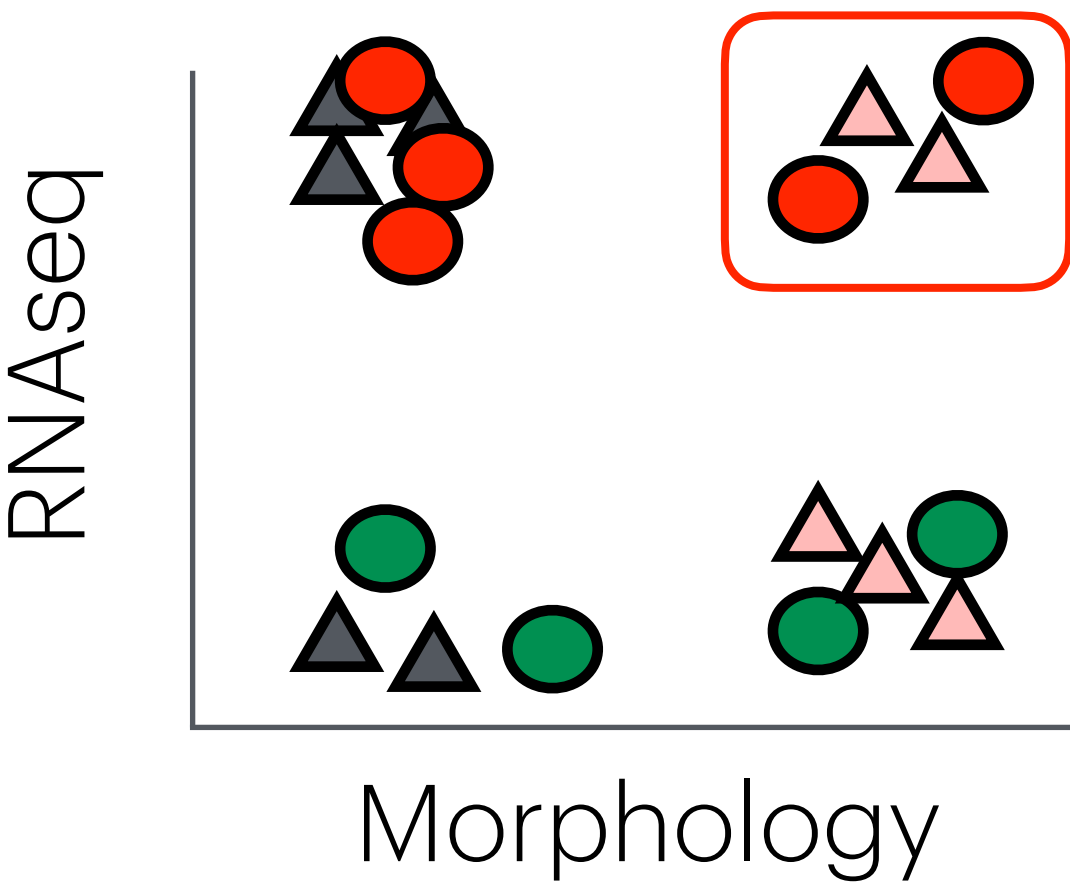
Motivation: **Example of Synergy**



Alignment



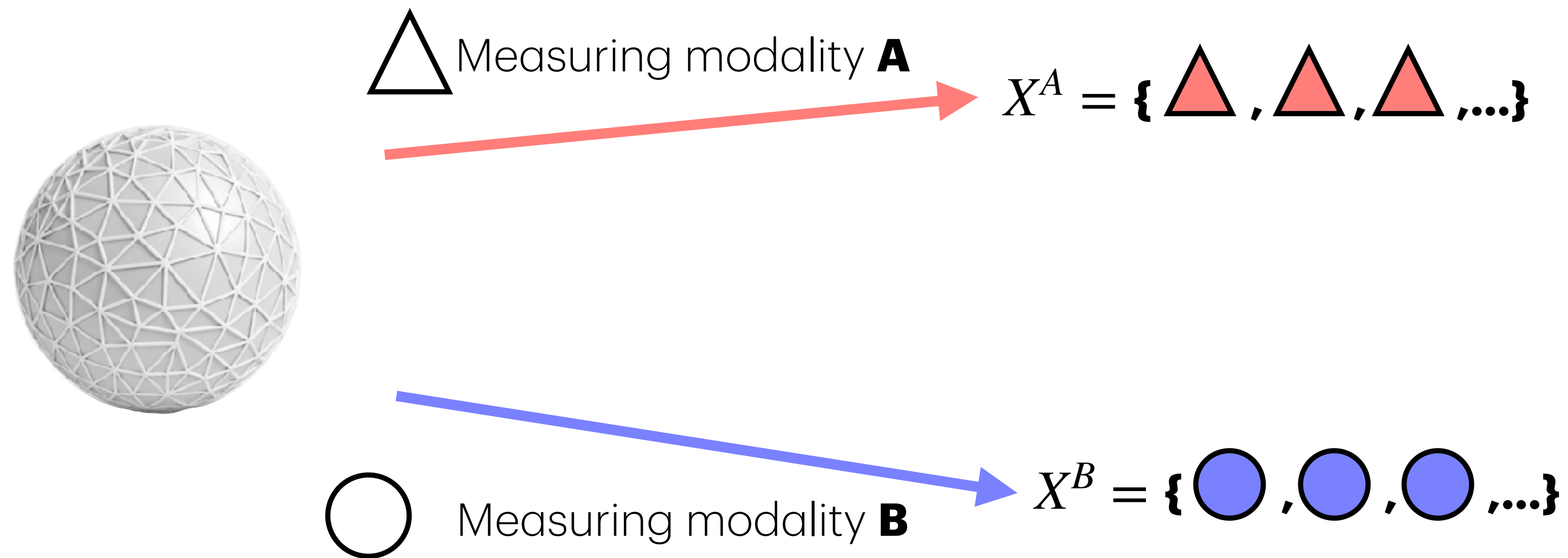
Synergy



Conceptual motivation: Serra, Matteo, et al. "Spatial transcriptomics reveals tumor microenvironment-driven subtypes of invasive lobular carcinoma." *Proceedings of the National Academy of Sciences* 123.6 (2026)

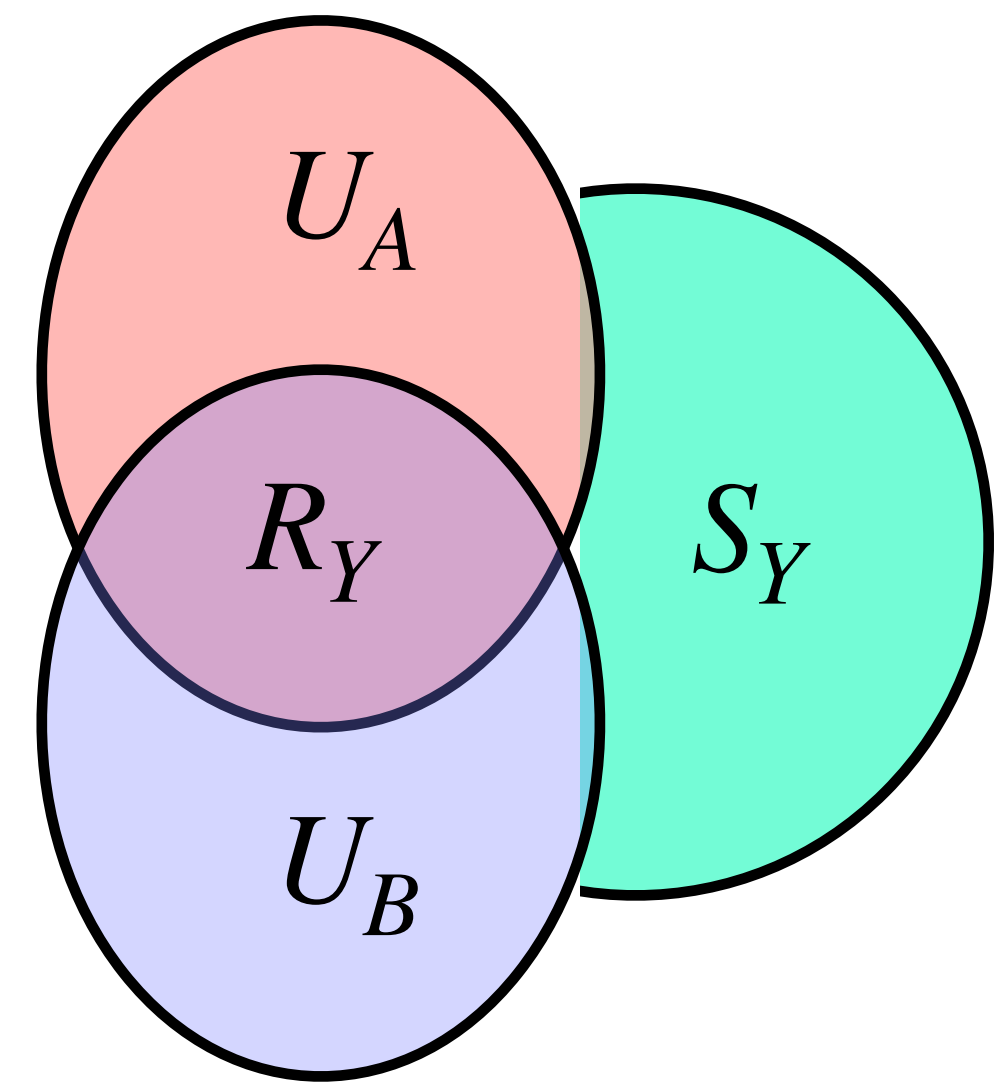
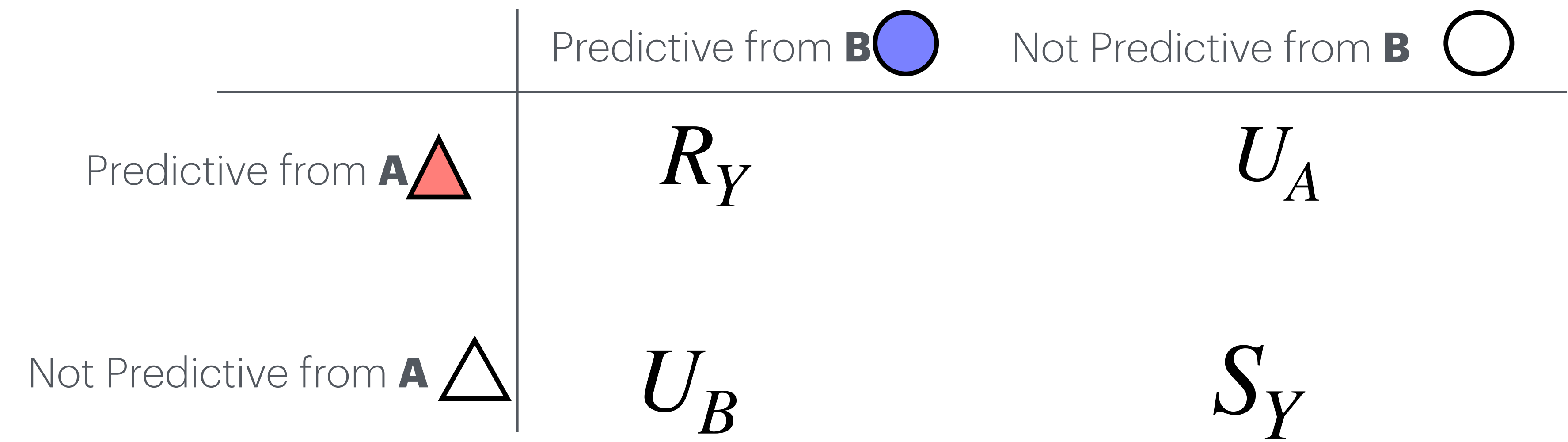
Motivation: What will we learn in this lecture

- Modalities A and B are two partial measurements of the same underlying system.
- Each modality may reveal information the other does not.
- Our question: does using both improve prediction of a target Y , beyond either modality alone?



Motivation: What will we learn in this lecture

Formally, what fusion is trying to solve?



In this lecture we study **fusion** in two settings:

- A) The supervised regime, because synergy is defined against a target label Y
- B) The label-free regime by reconstruction and generative models*

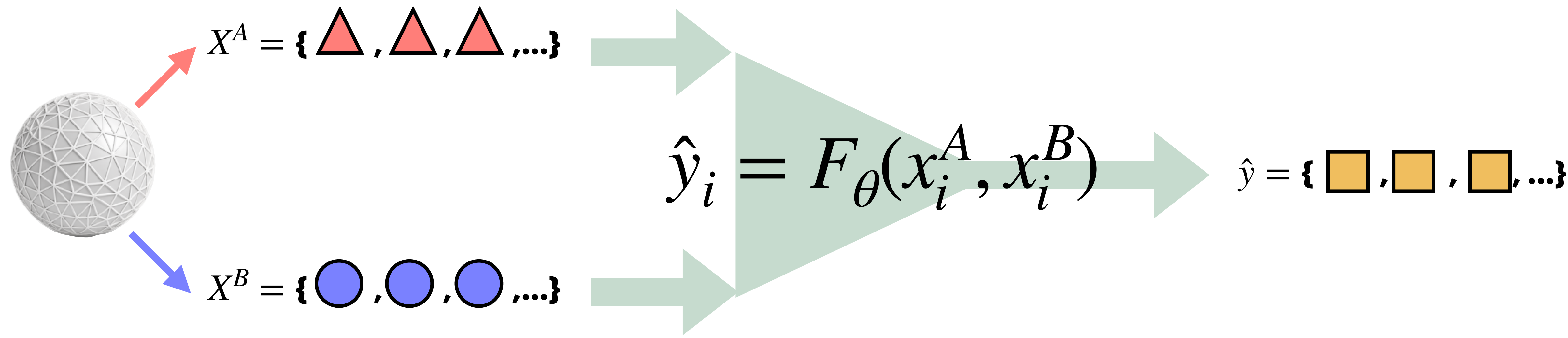
***Note:** alignment in previous lecture was already label-free; here the signal is reconstruction, not correspondence

Part A: The supervised multimodal setup

Fusion as a supervised learning problem over paired modalities and a target: $(x_i^A, x_i^B) \rightarrow \hat{y}_i$.

For each paired observation, we want to learn: $\hat{y}_i = F_{\theta}(x_i^A, x_i^B)$

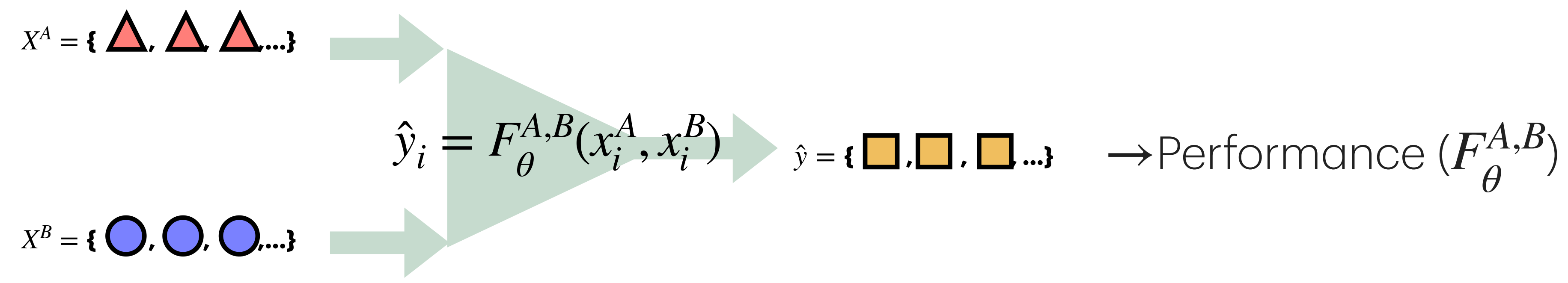
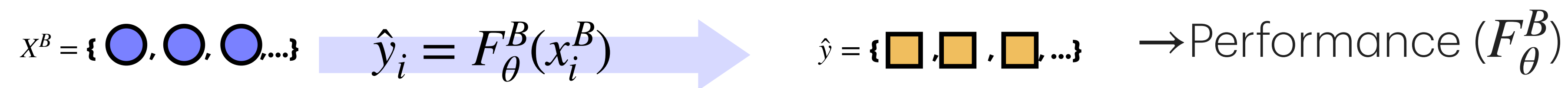
We should achieve this by minimizing the loss: $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i)$



Part A: What counts as successful fusion?

If successful, the multimodal model should perform better than both of unimodal models:

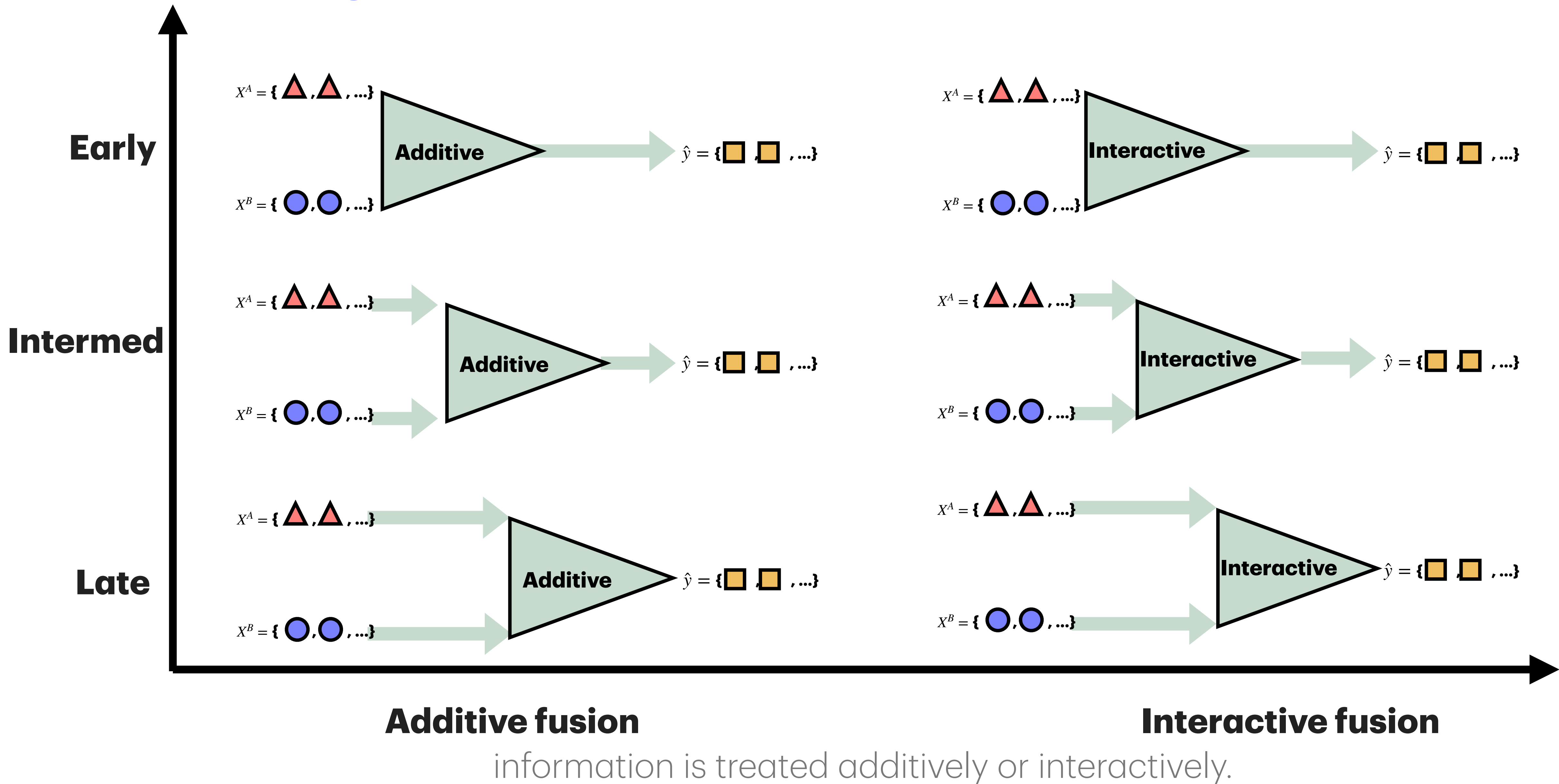
Performance ($F_{\theta}^{A,B}$) **> max{ Performance** (F_{θ}^A) **, Performance** (F_{θ}^B) **}**



We will revisit this point later...

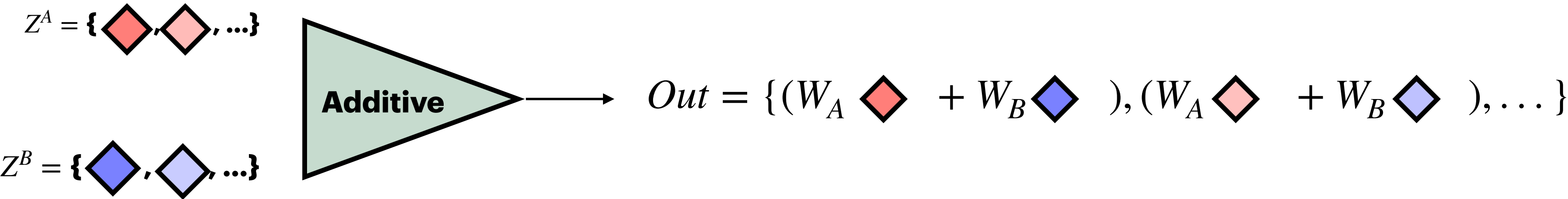
Part A: The fusion design space - Where modalities meet and how they interact

where *A* and *B* first meet

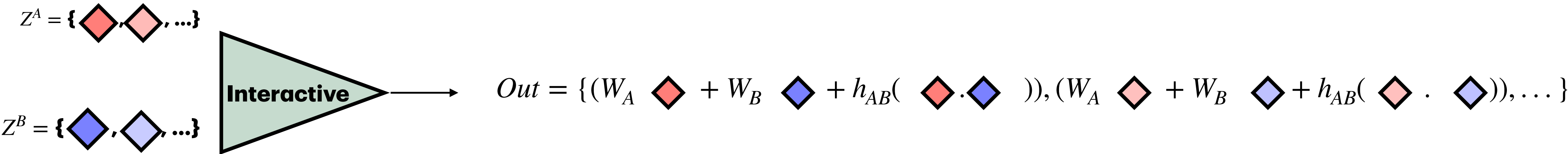


Part A: What is additive and what is interactive?

Explicitly Additive, changing Z^A does not affect how Z^B is weighted*: $Out = W_A z^A + W_B z^B$



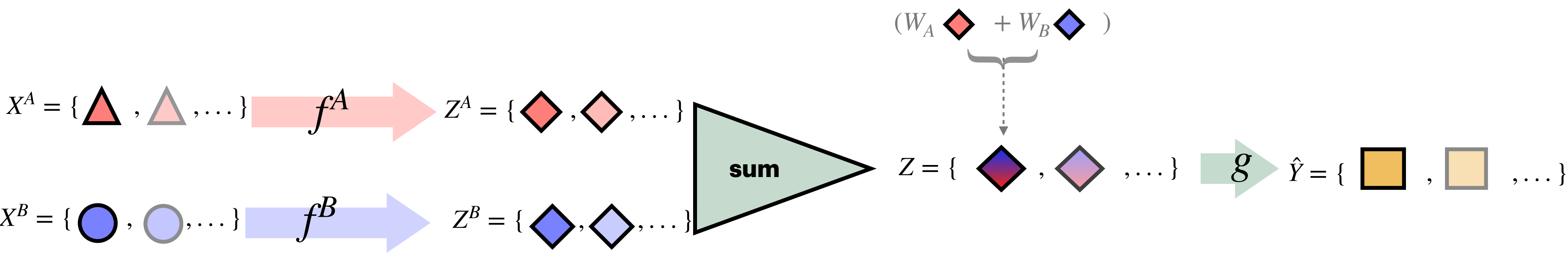
Explicitly Interactive, changing Z^A does affect how Z^B is weighted: $Out = W_A z^A + W_B z^B + h_{AB}(z^A, z^B)$



Note*: W_A and W_B are generally matrices, so that $W_A z^A$ and $W_B z^B$ map to the same space and hence the additive

Part A: The simplest additive fusion, late sum fusion

Additive Fusion: Each modality makes its own contribution to the prediction; the model adds those contributions to perform final prediction $\hat{Y}$



$$z_i^A = f^A(x_i^A)$$

$$z_i^B = f^B(x_i^B)$$

$$z_i = W_A z_i^A + W_B z_i^B$$

$$\hat{y}_i = g(z_i), g \text{ is a small and simple prediction head}$$

Part A: Late additive fusion, example

Late fusion = each modality predicts alone; a weighted sum fuses the scores.

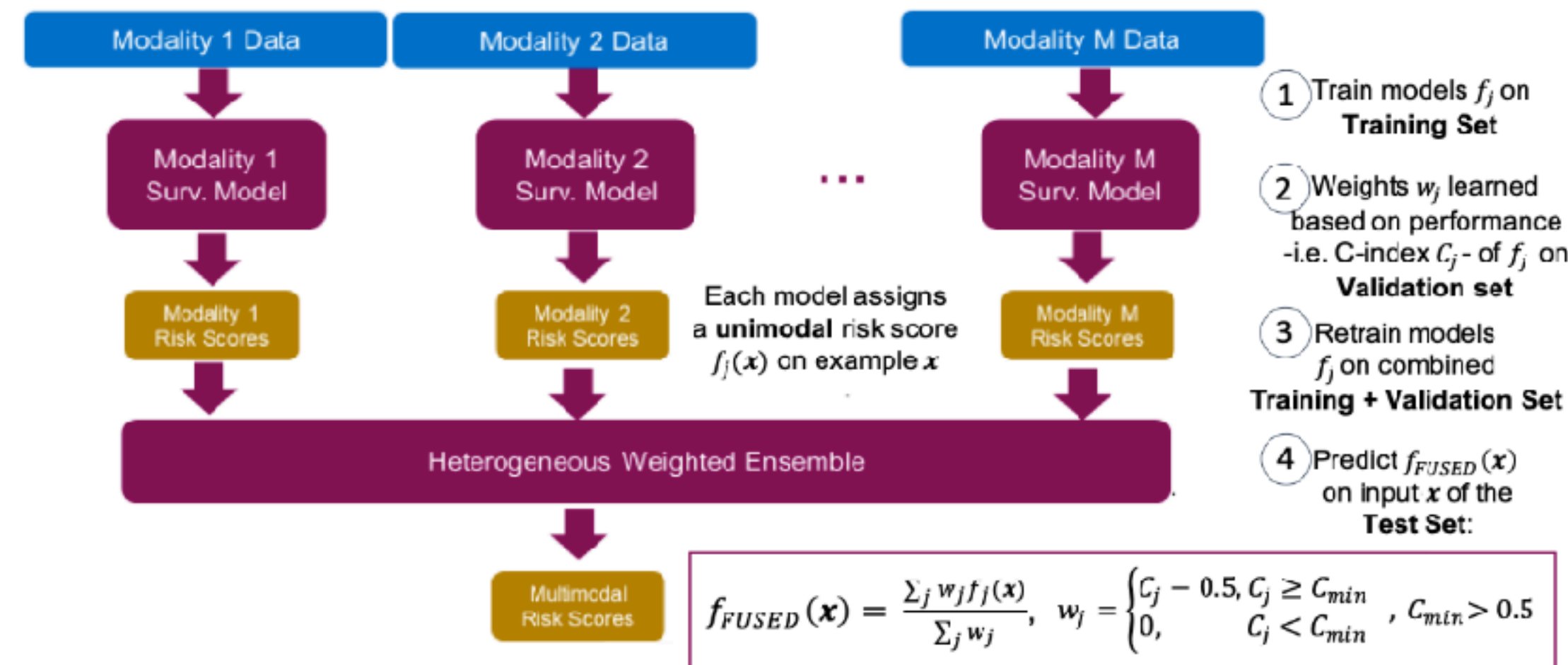
Real gain over the best unimodal is modest, but grows with sample size.

Additive late fusion is known to be fault-tolerant, important for safety measures.

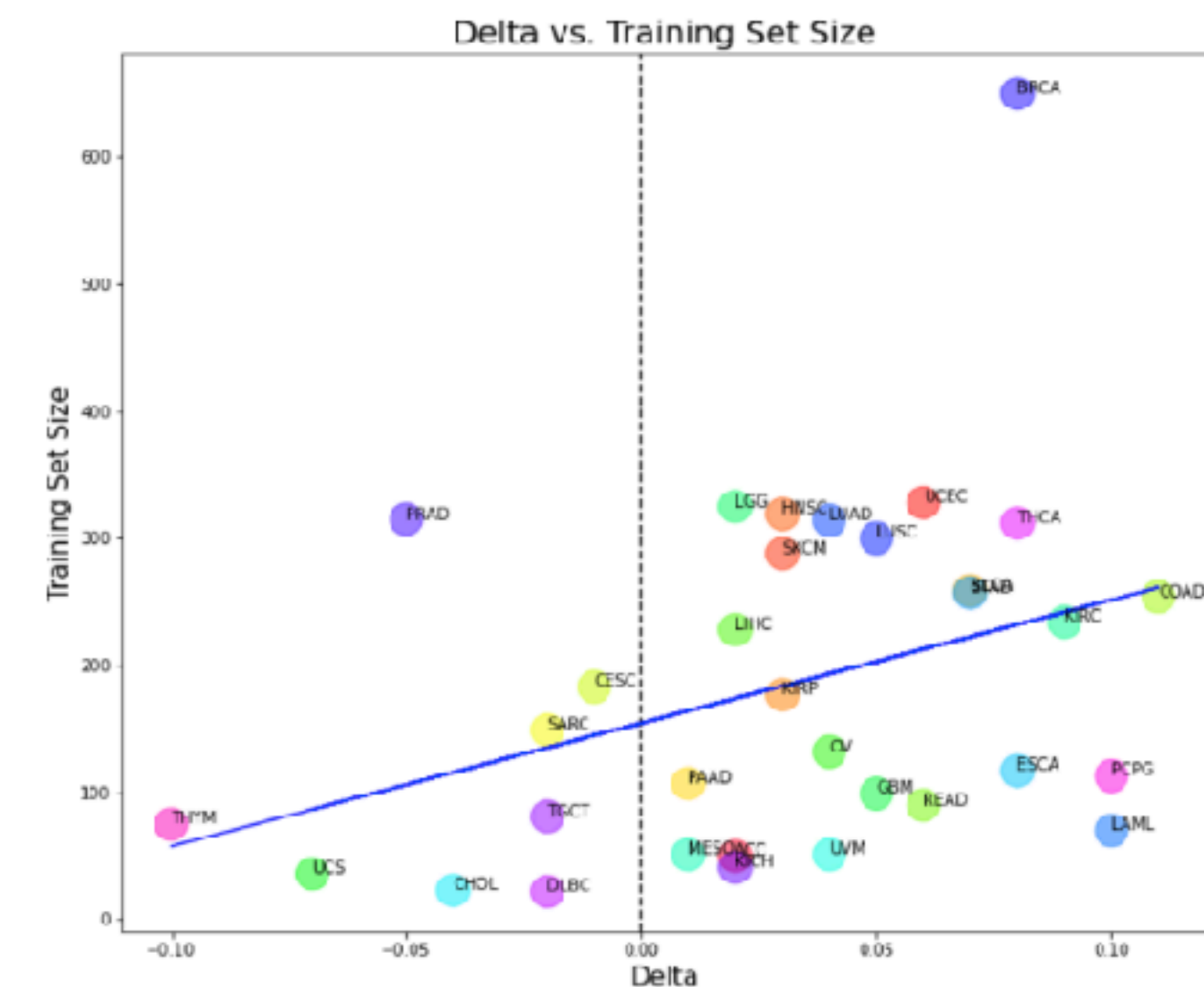
Modalities

- clinical/demographic
- gene expression
- DNA mutations METH
- DNA methylation
- microRNA expression
- protein expression
- long non-coding RNA expression

Architecture


$$\text{Delta} = (\text{multimodal performance} - \text{best-unimodal performance})$$

Result



But Late additive fusion cannot learn about interactions in the data easily...

Part A: Late additive fusion, it can fail as jointly-trained encoders compete

Modality Competition: What Makes Joint Training of Multi-modal Network Fail in Deep Learning? (Provably)

Yu Huang¹ Junyang Lin² Chang zhou² Hongxia Yang² Longbo Huang¹

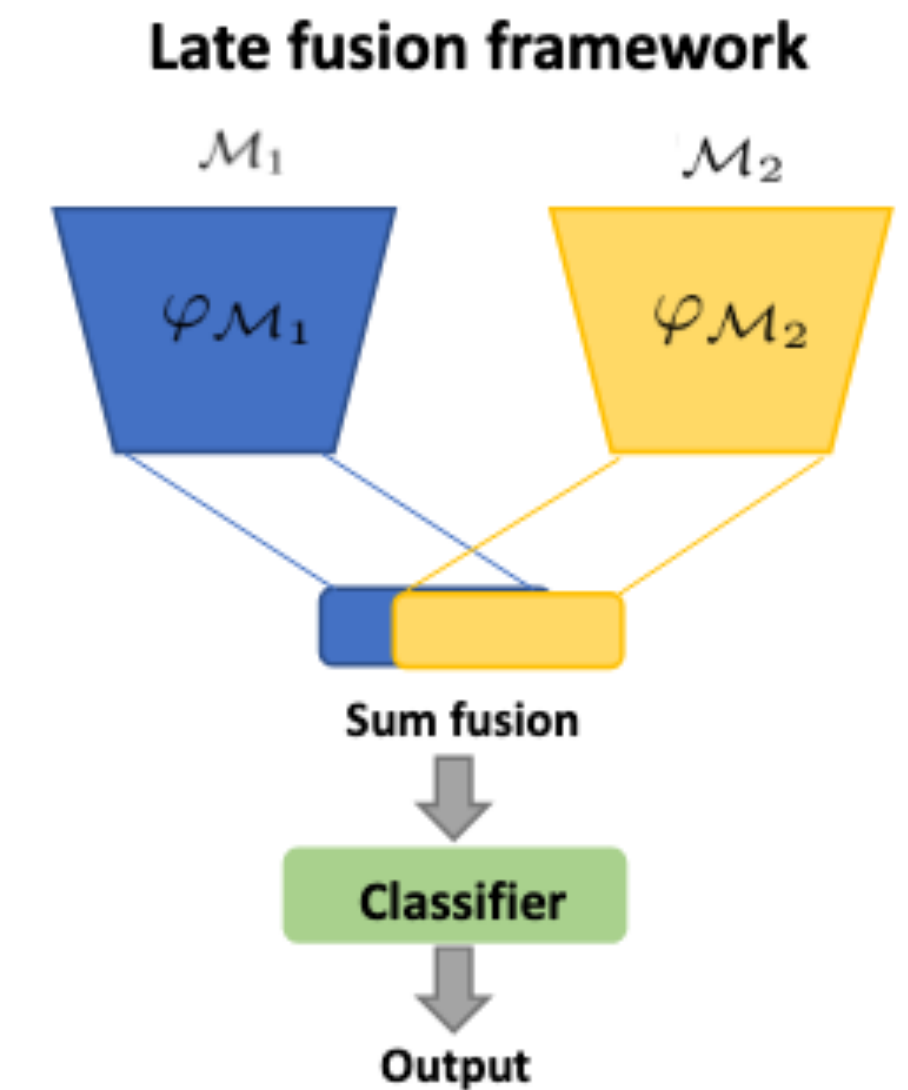
ICML 2022

Fundamental Questions

1. How does the neural network encoder of each modality, trained by multi-modal learning, learn its feature representation?
2. Why does multi-modal learning in deep learning collapse in practice when naive joint training is applied?

Modality Competition

During joint training, multiple modalities will compete with each other. Only a *subset* of modalities which correlate more with their encoding network's random initialization will win and be learned by the final network with other modalities failing to be explored.



How to improve the performance?

A) separate supervision for each encoder

B) Use pretrained encoders

C) control each modality's participation during training by gradient modulation

(Peng, Xiaokang, et al. 2022.)

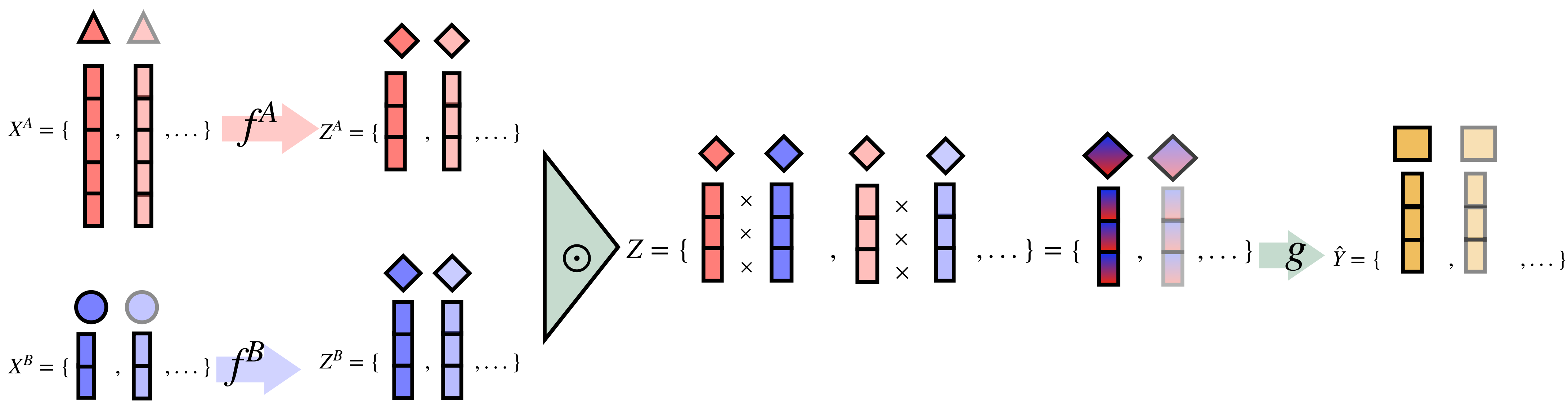
Part B: The smallest interactive fusion, element-wise multiplicative $\odot$ fusion

Core idea: First map both modality embeddings into the same interaction space, Then multiply matching coordinates:

$$z_i^A = f^A(x_i^A)$$

$$z_i^B = f^B(x_i^B)$$

$$z = z_i^A \odot z_i^B$$



Or more generally: $z_i = W_A z_i^A + W_B z_i^B + W_{\odot} (z_i^A \odot z_i^B)$

Part B: example of element-wise multiplicative $\odot$ fusion

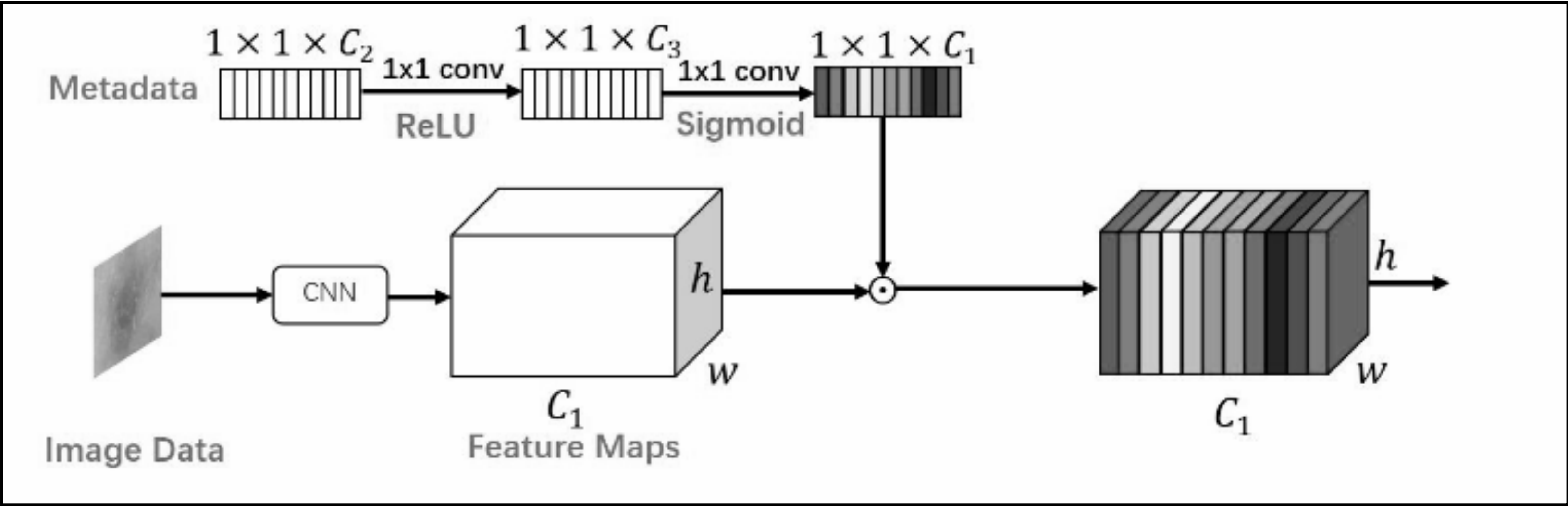


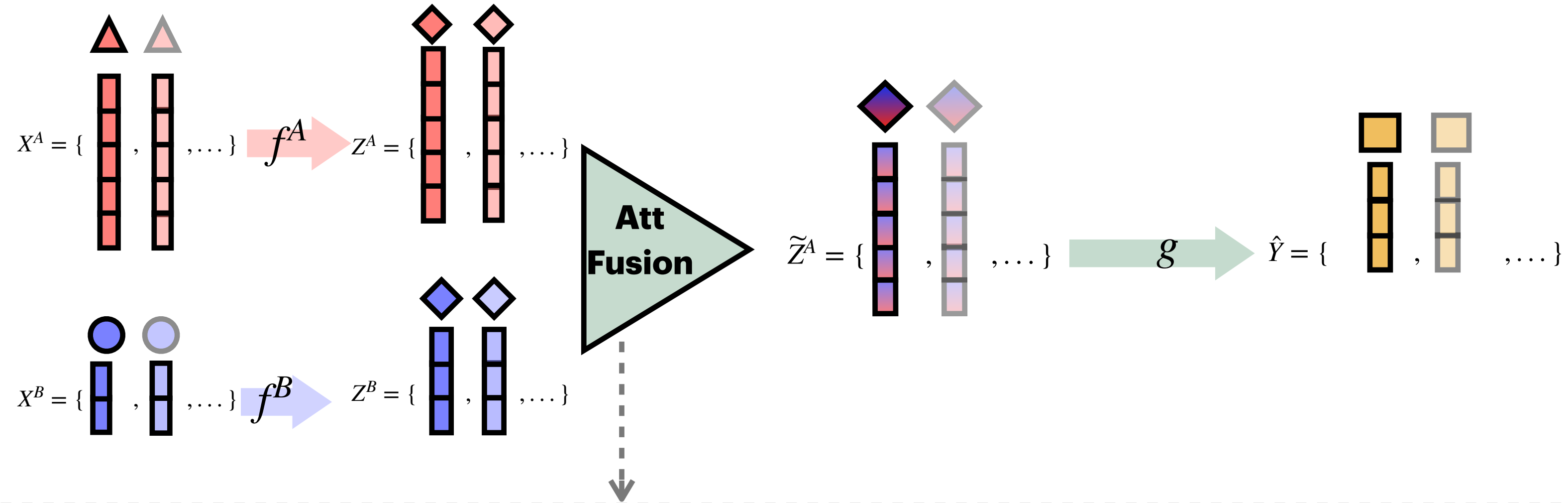
Table 1. Comparison of the proposed multiplication-based fusion approach with two baseline approaches on multiple backbone CNN architectures.

Backbones	No metadata	Concatenation-based	Ours
AlexNet	74.68 ± 0.92	76.55 ± 1.25	78.26 ± 1.55
VGG19	81.60 ± 1.67	82.35 ± 1.68	84.06 ± 1.16
ResNet50	82.50 ± 1.31	82.98 ± 1.35	84.02 ± 1.50
DenseNet161	84.59 ± 1.42	85.85 ± 0.92	87.03 ± 1.40
SENet154	85.44 ± 1.09	86.46 ± 0.69	87.64 ± 0.52
PNASNet-5	87.90 ± 1.32	87.25 ± 0.73	89.09 ± 0.67

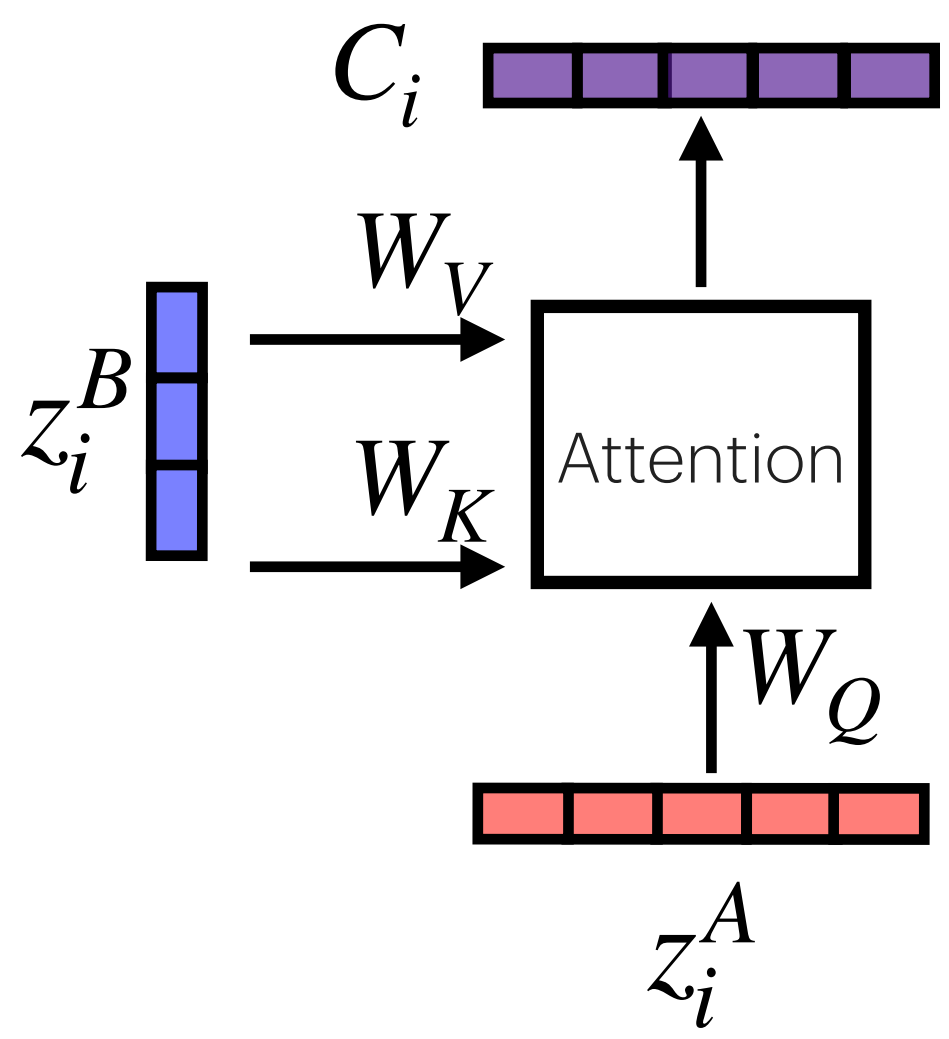
Caveat: only matching coordinates interact...

$$Z = \left\{ \begin{array}{c} \text{Red Diamond} \\ \text{Red Column} \end{array} \times \begin{array}{c} \text{Blue Diamond} \\ \text{Blue Column} \end{array}, \begin{array}{c} \text{Light Red Diamond} \\ \text{Light Red Column} \end{array} \times \begin{array}{c} \text{Light Blue Diamond} \\ \text{Light Blue Column} \end{array}, \dots \right\} =$$

Part B: Cross-attention fusion, each element of A can read from every element of B



For each element of z_i^A , cross-attention compares it with every element of z_i^B , uses those comparisons to build a weighted summary of useful information from z_i^B . We will then add that summary back to z_i^A to build $\tilde{z}_i^A$

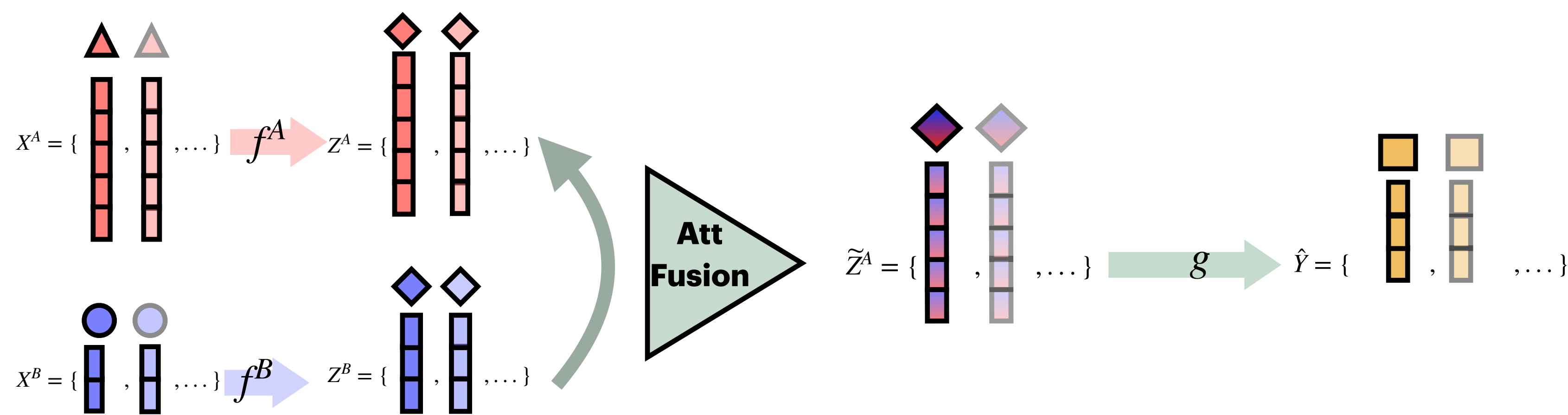


$$Q_i^A = z_i^A W_Q$$
$$K_i^B = z_i^B W_K$$
$$V_i^B = z_i^B W_V$$
$$c_i^{A \leftarrow B} = \text{softmax}_{\text{row}} \left(\frac{Q_i^A (K_i^B)^T}{\sqrt{d_K}} \right) V_i^B$$

Then

$$\tilde{z}_i^A = z_i^A + c_i^{A \leftarrow B}$$

Part B: Cross-attention fusion, each element of A can read from every element of B



Note that: Cross-attention does not collapse Z^A and Z^B into one anonymous fused vector. It updates the representation of Z^A using information selectively read from Z^B , and that is why we called the final product $\tilde{Z}^A$ and not Z .

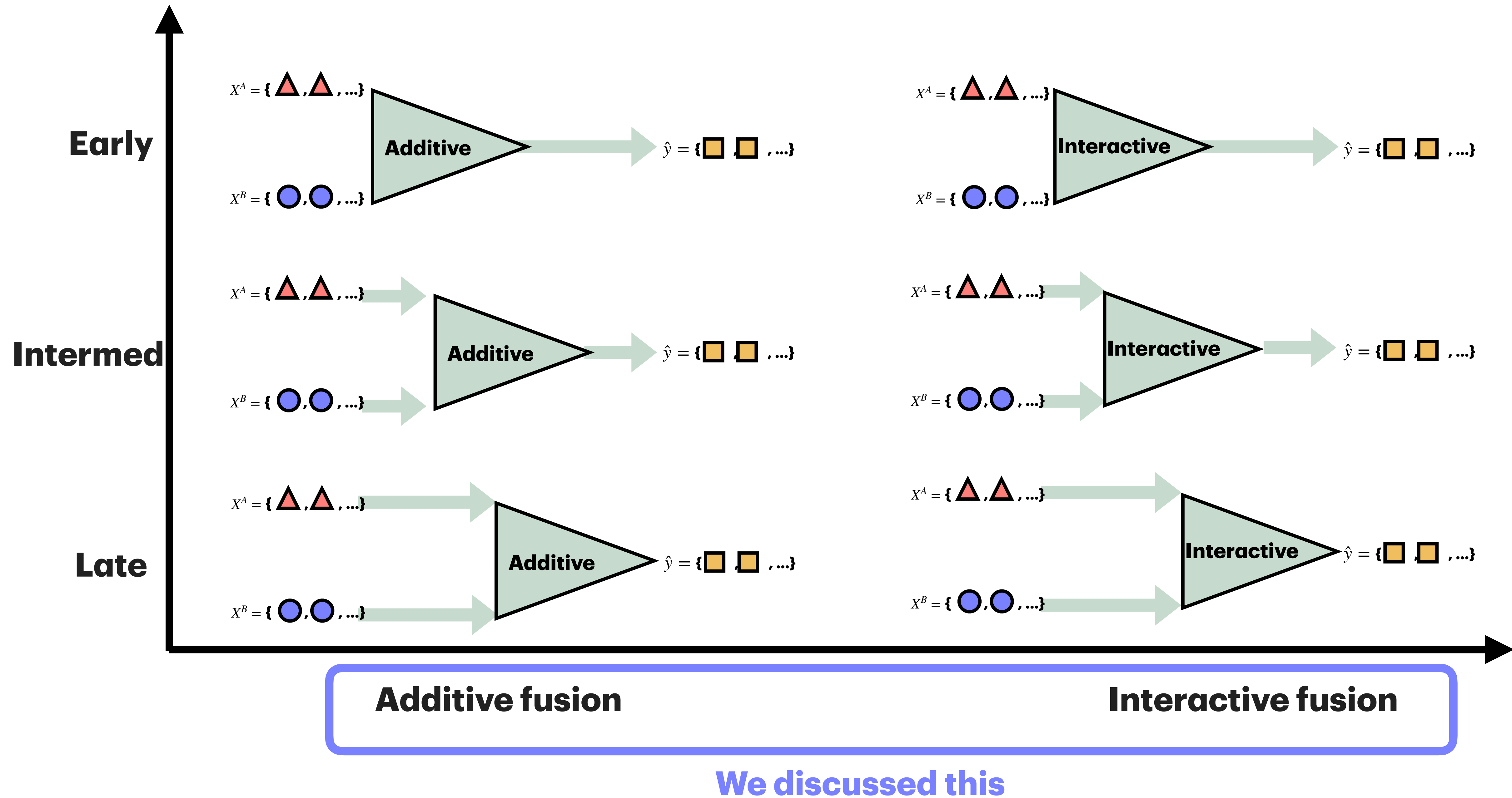
$$z_i^A \rightarrow Q_i^A$$

$$z_i^B \rightarrow K_i^B, V_i^B$$

$$c_i^{A \leftarrow B} = \text{softmax} \left(\frac{Q_i^A (K_i^B)^\top}{\sqrt{d_k}} \right) V_i^B$$

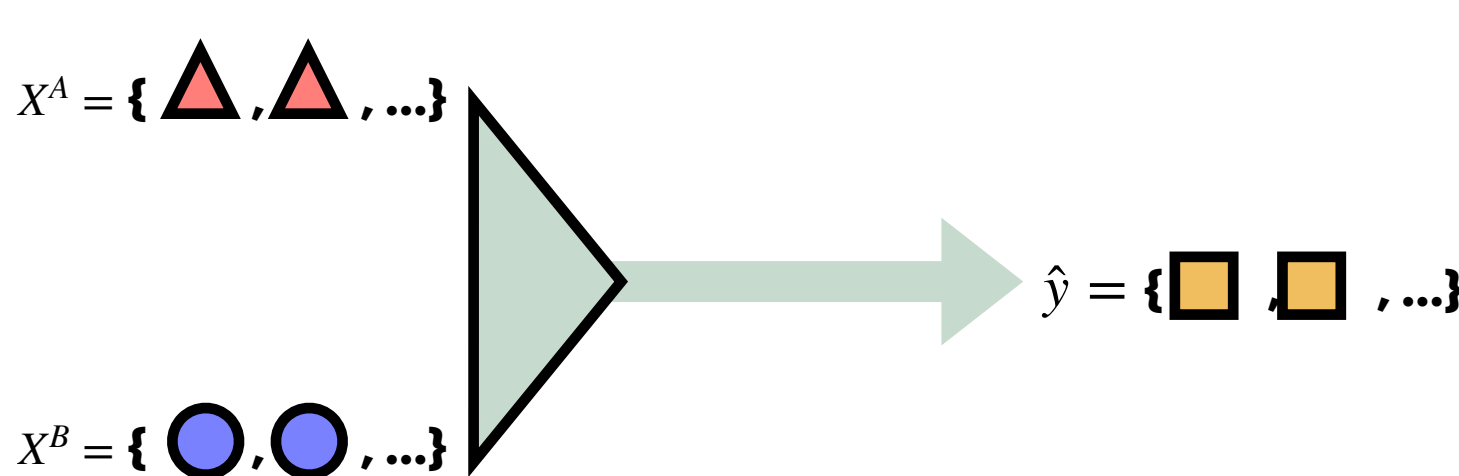
$$\tilde{z}_i^A = z_i^A + c_i^{A \leftarrow B}.$$

where A and B first meet



Part B: Fusion timing, when should modalities interact?

Early Fusion



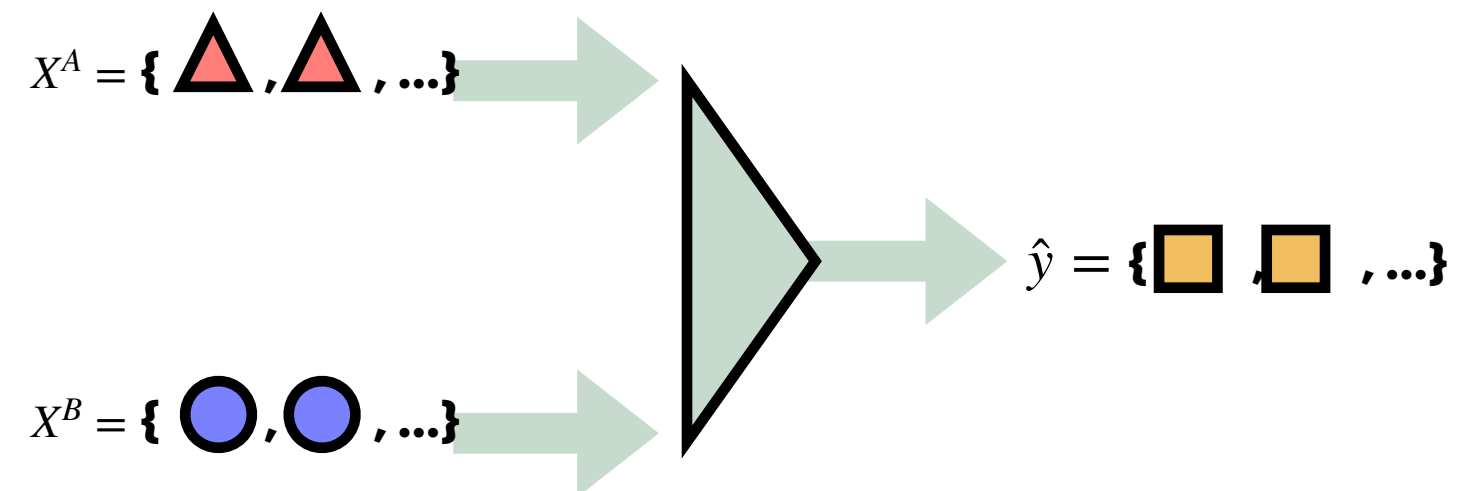
Pros

- +Can model S_Y from raw inputs has
- +maximum joint flexibility

Cons

- Needs the most paired data
- Different scales can clash
- Missing input breaks the path

Intermediate Fusion



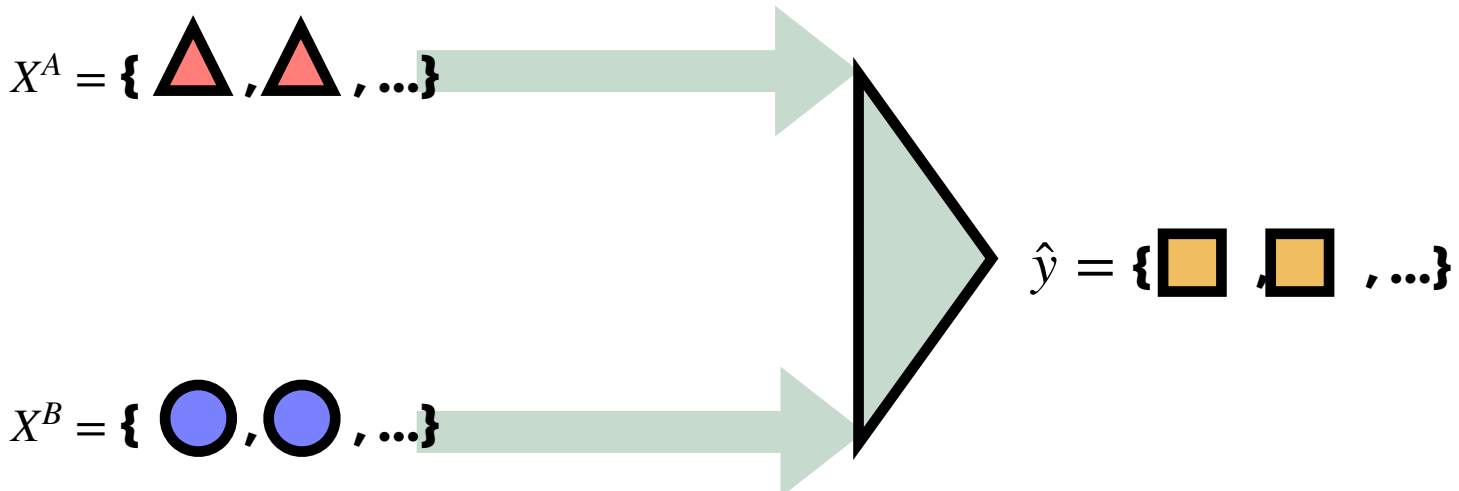
Pros

- +Can model S_Y on learned features
- +Encoders can be pretrained or frozen
- +Often easier to train than early fusion

Cons

- Missing modality can disrupt it unless it is trained for it

Late Fusion

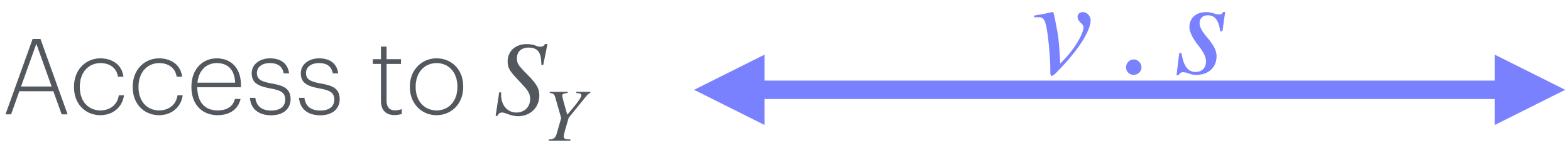


Pros

- +Separate, auditable predictors
- +Easiest to degrade with missing modality
- +Can down-weight a poor modality

Cons

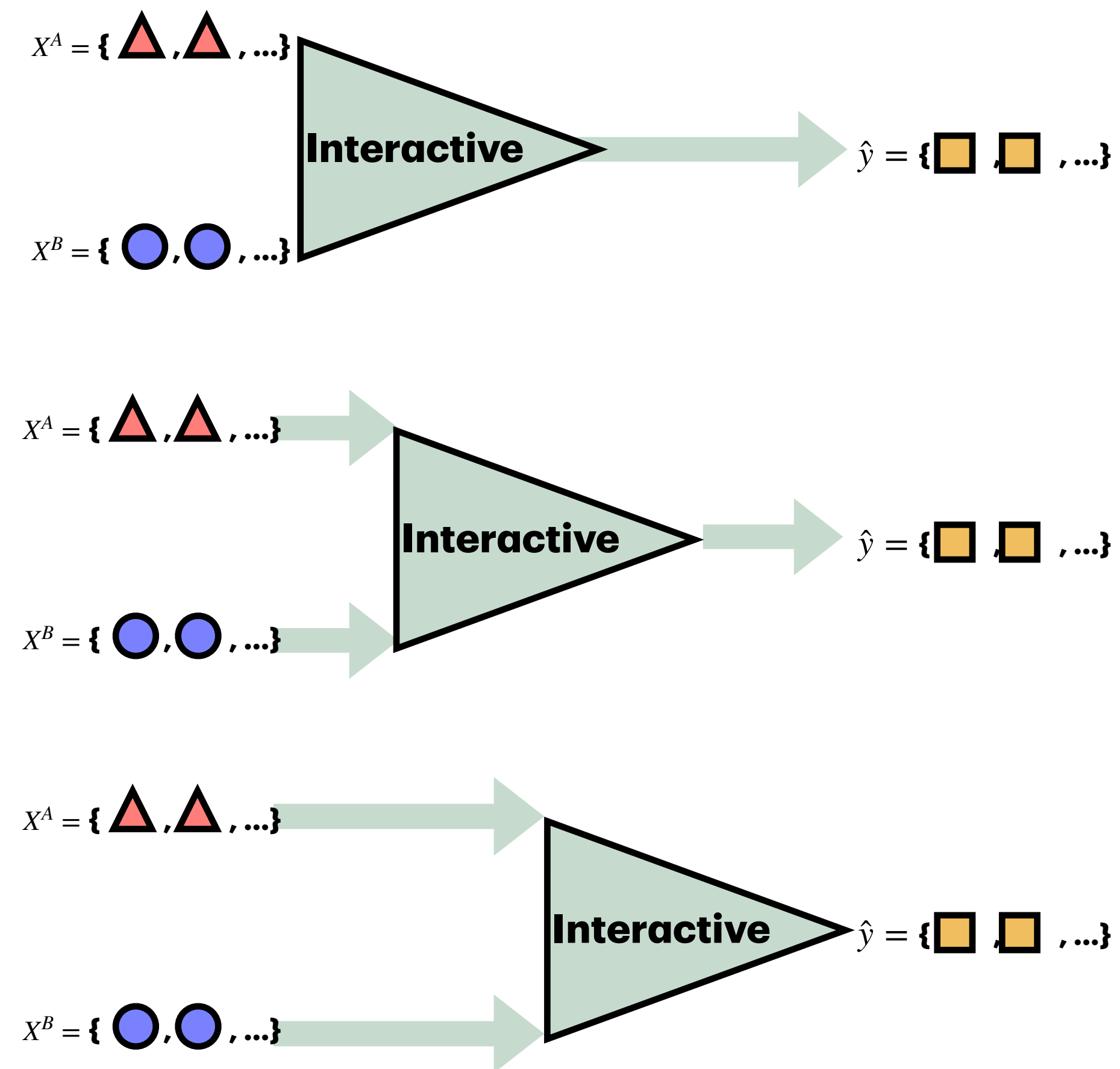
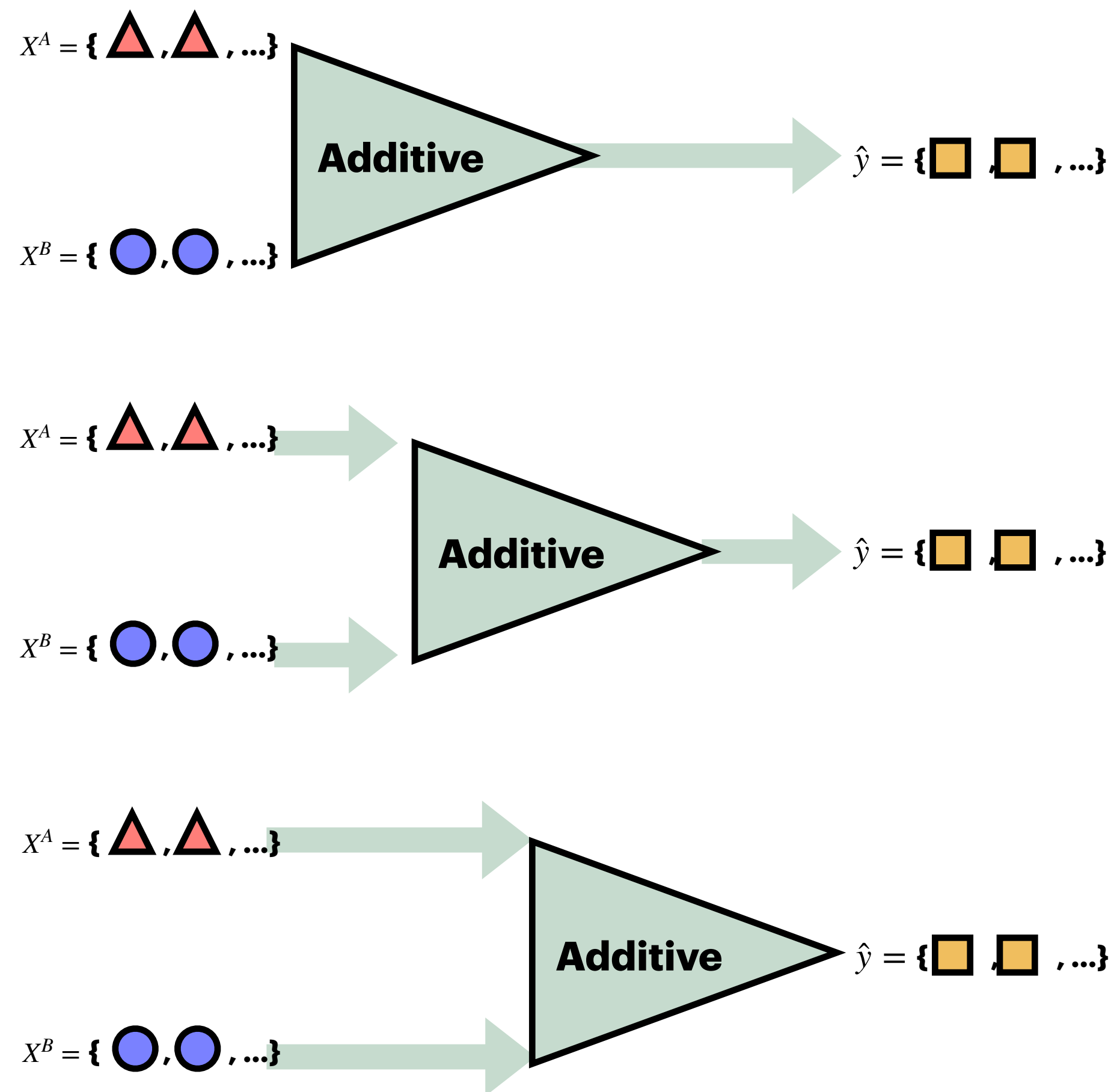
- Cannot recover feature-level S_Y well
- Often trades joint signal for robustness



Modularity
Missing-modality robustness.

where A and B first meet

Early
Intermed
Late



Additive fusion

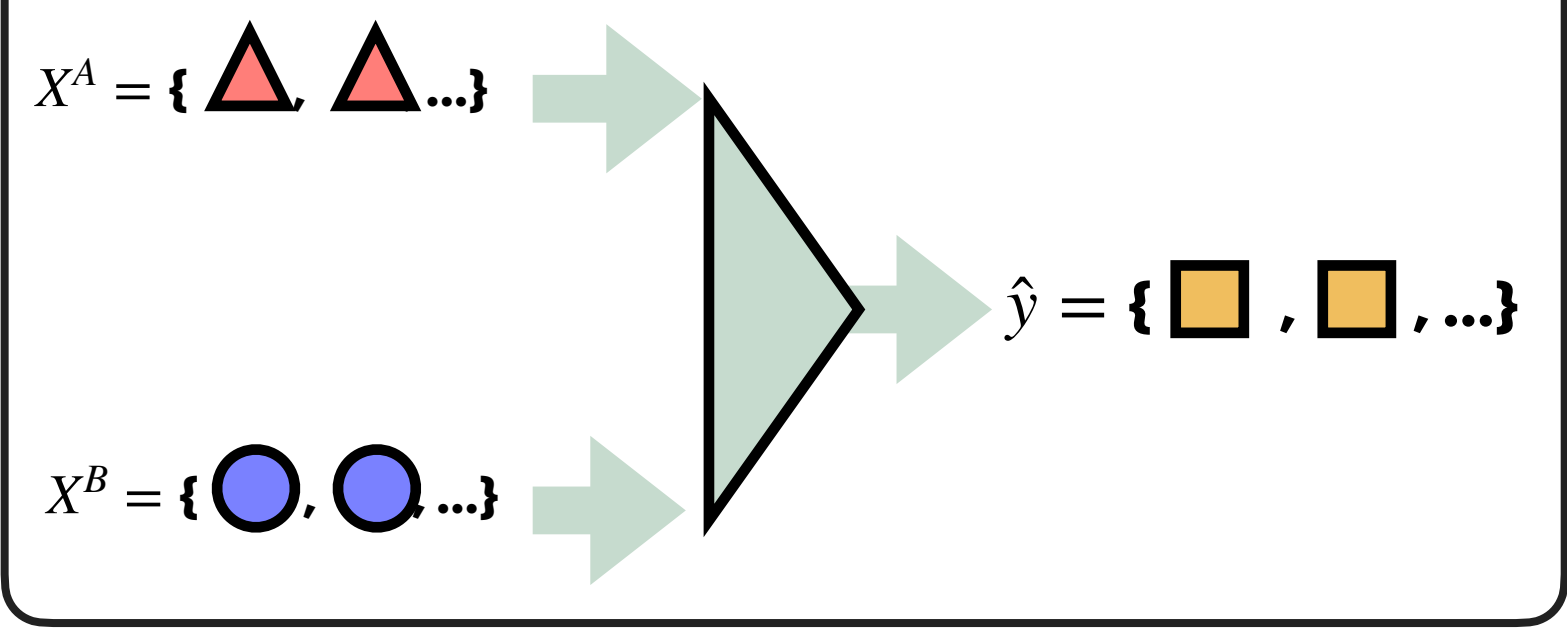
Interactive fusion

We discussed these

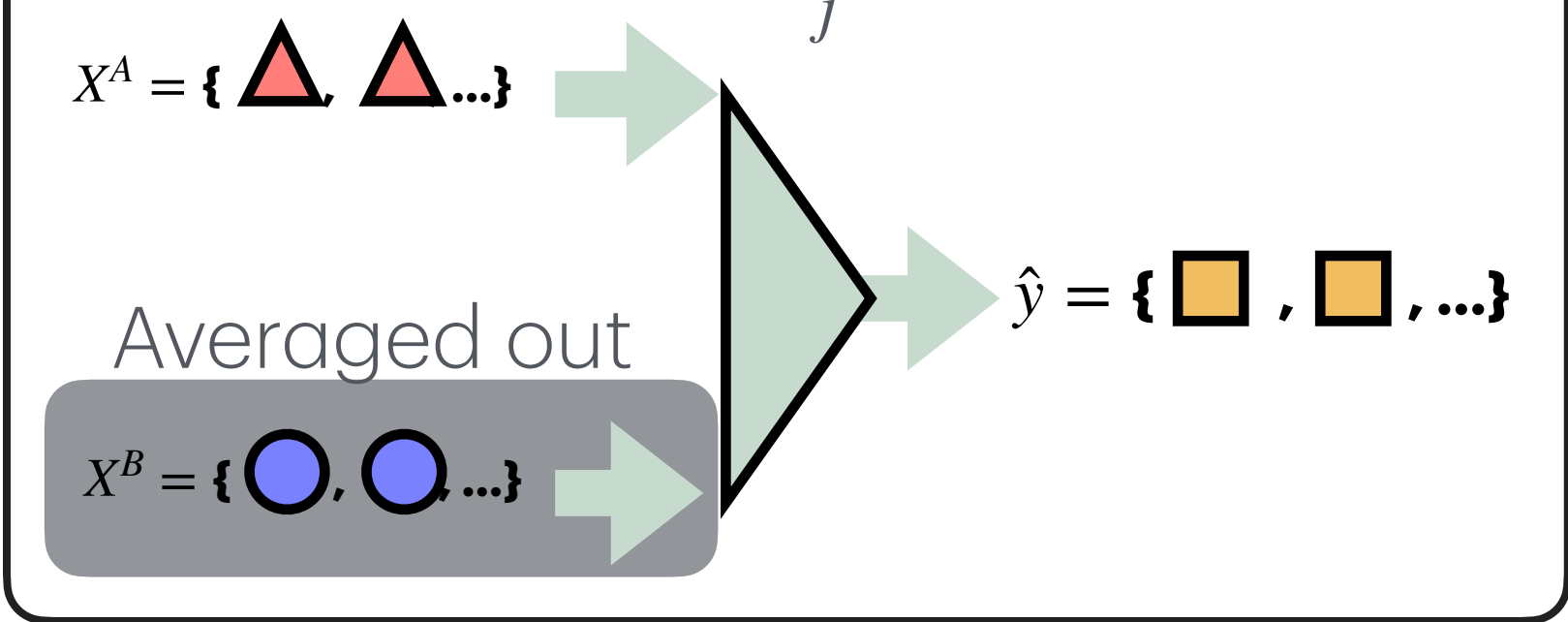
Part B: We learned about Fusion, but did the model actually use cross-modal interaction?

We can ask this question rigorously with the setup below:

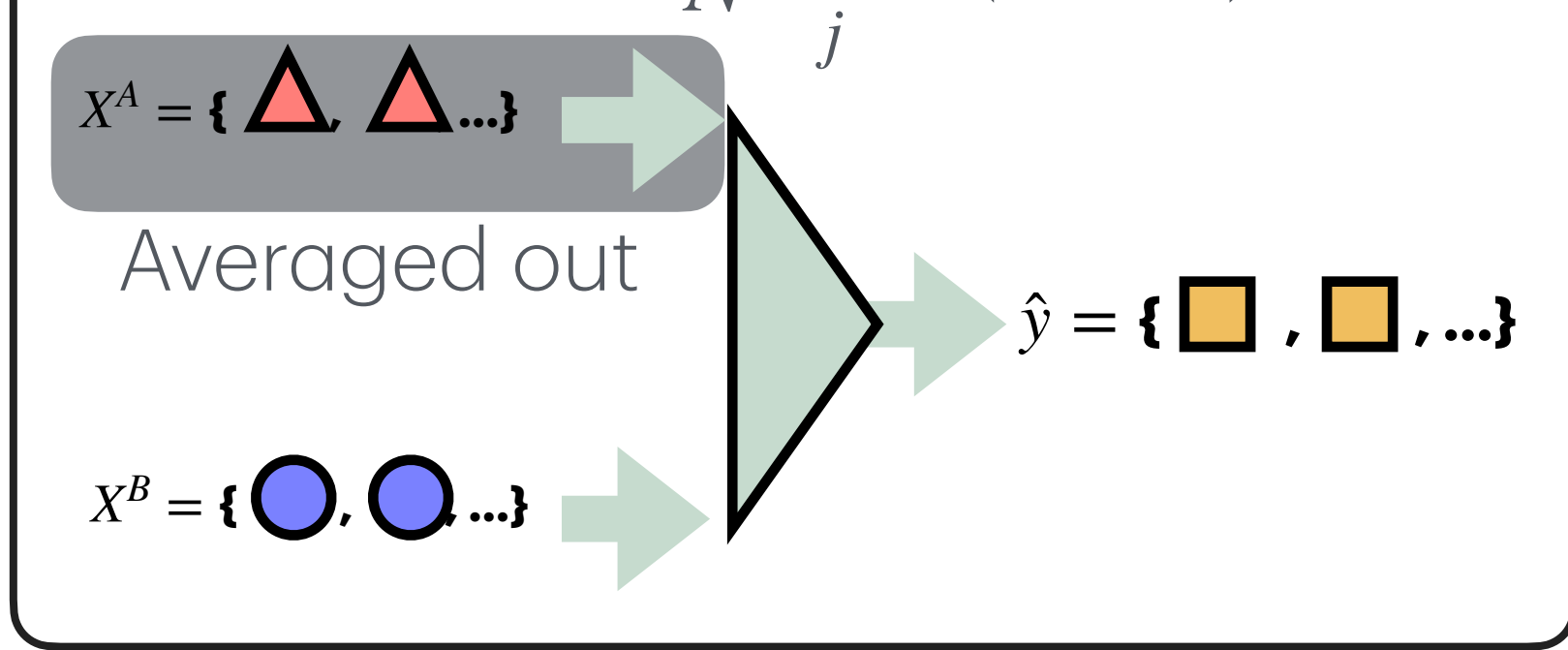
1. Our full multimodal model is:

$$\hat{y}_i = f(x_i^A, x_i^B)$$


2. Contribution of *A* by averaging *B* out in each prediction:

$$\bar{f}_A(x_i^A) = \frac{1}{N} \sum_j f(x_i^A, x_j^B)$$


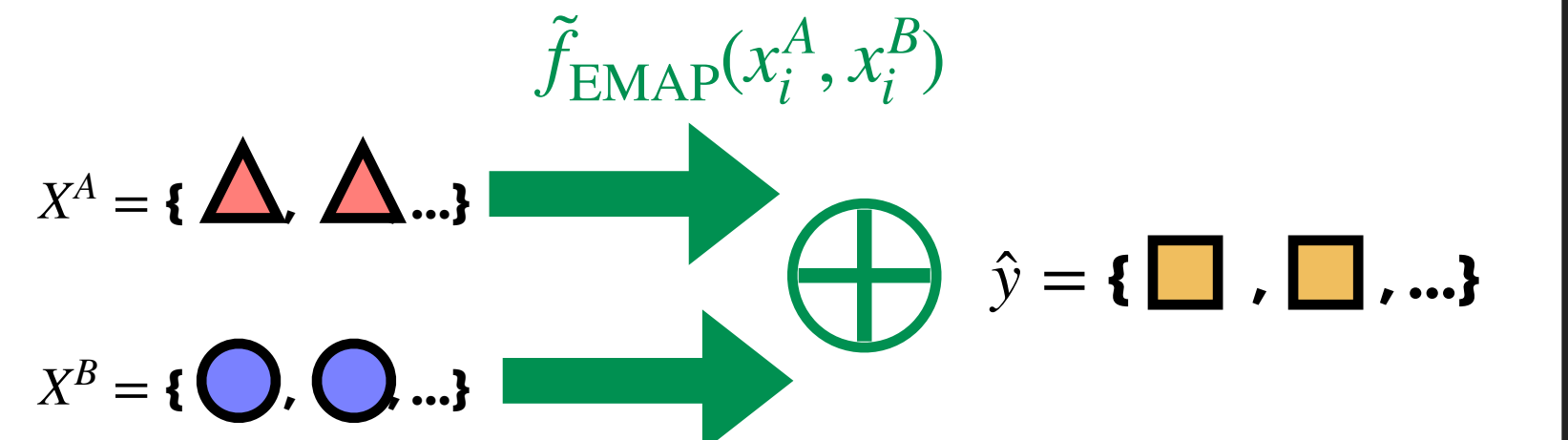
3. Contribution of *B* by averaging *A* out in each prediction:

$$\bar{f}_B(x_i^B) = \frac{1}{N} \sum_j f(x_j^A, x_i^B)$$


4. Now we can compute the Empirical **Multimodally-Additive** Projection (EMAP) as:

$$\tilde{f}_{\text{EMAP}}(x_i^A, x_i^B) = \bar{f}_A(x_i^A) + \bar{f}_B(x_i^B) - \mu$$

$\mu = \frac{1}{N^2} \sum_{i,j} f(x_i^A, x_j^B)$ removes the overall baseline that appears in both averaged contributions.



$\tilde{f}_{\text{EMAP}}(x_i^A, x_i^B)$ is the best empirical additive approximation to the fixed model $f(x_i^A, x_i^B)$!

Part B: What are some challenges? How can we address them

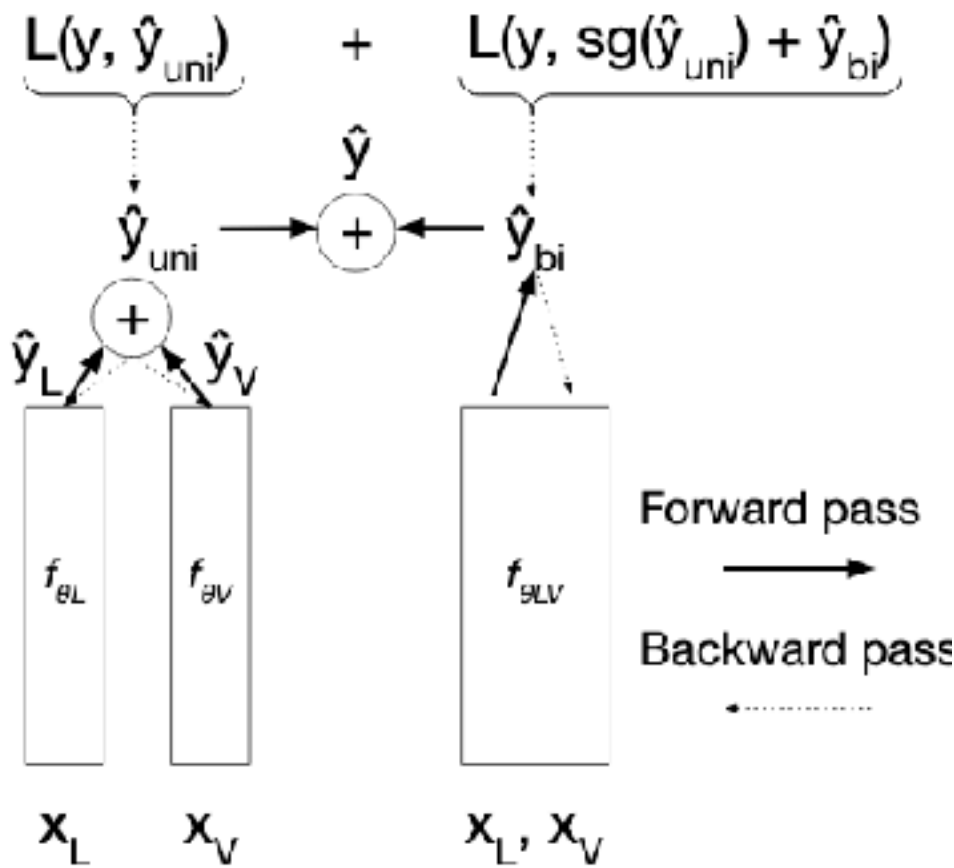
Interactive model ≈ its own additive (EMAP)

→ the interaction barely helped

	I-INT	I-SEM	I-CTX	T-VIS	R-POP	T-ST1	T-ST2
Metric	AUC	AUC	AUC	Weighted F1	ACC	AUC	ACC
Cross-val Setup	5-fold	5-fold	5-fold	10-fold	15-fold	5-fold	5-fold
Constant Pred.	50.0	50.0	50.0	17.2	50.0	50.0	66.2
Prev. SOTA	85.3	69.1	78.8	44	62.7	N/A	70.5
Neural Network (I)	90.4	69.2	78.5	51.1	63.5	71.1	79.9
Polykernel SVM (I)	91.3	74.4	81.5	50.8	—	72.1	80.9
FT LXMERT (I)	83.0	68.5	76.3	53.0	63.0	66.4	78.6
↳ + Linear Logits (I)	89.9	73.0	80.7	53.4	64.1	75.5	80.3
Linear Model (A)	90.4	72.8	80.9	51.3	63.7	75.6	76.1
Our Best Interactive (I)	91.3	74.4	81.5	53.4	64.2	75.5	80.9
↳ + EMAP (A)	91.1	74.2	81.3	51.0	64.1	75.9	80.7

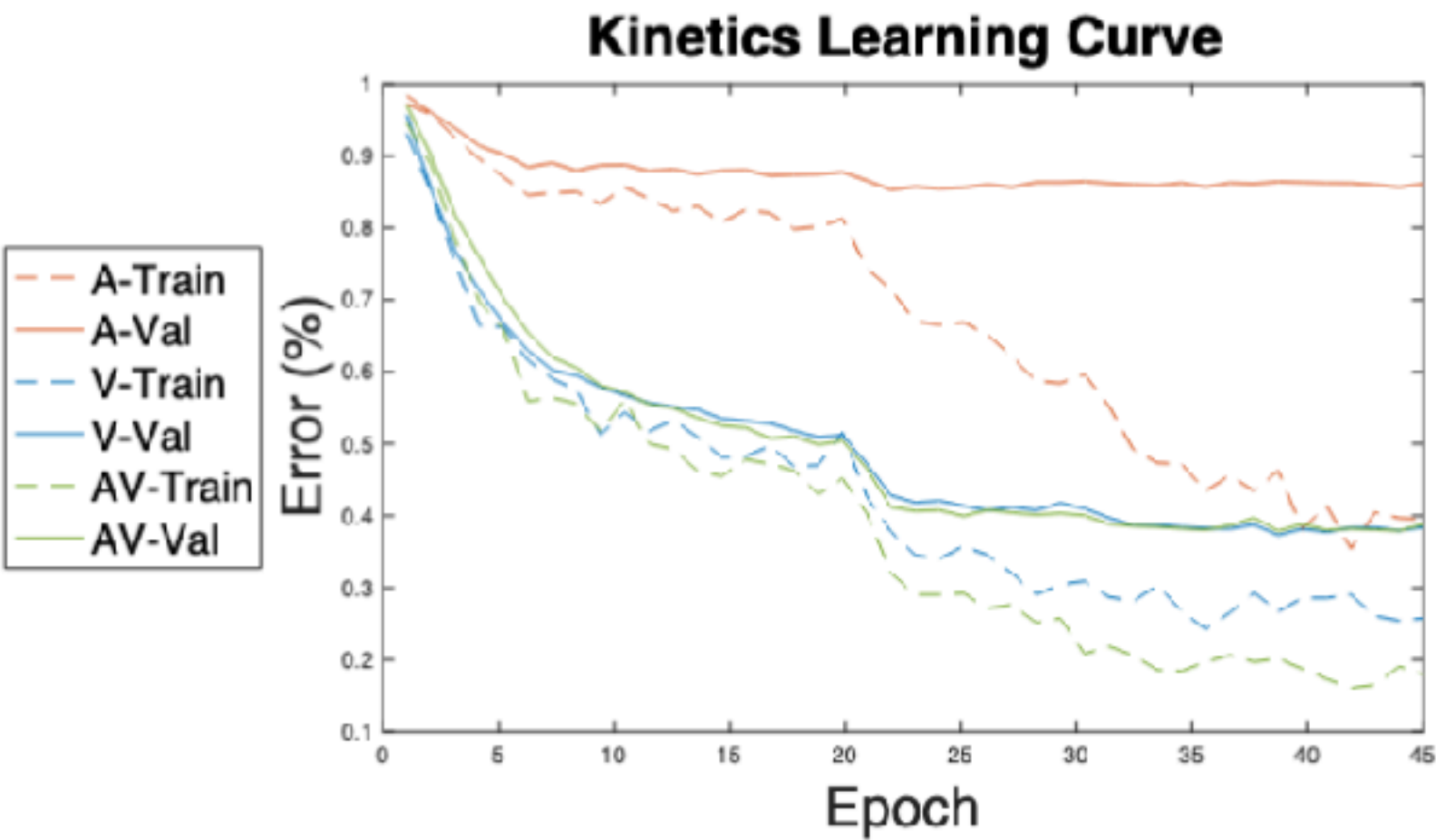
Hessel et al., Does my multimodal model learn cross-modal interactions?
It's harder to tell than you might think! EMNLP 2020

Residual training can help:

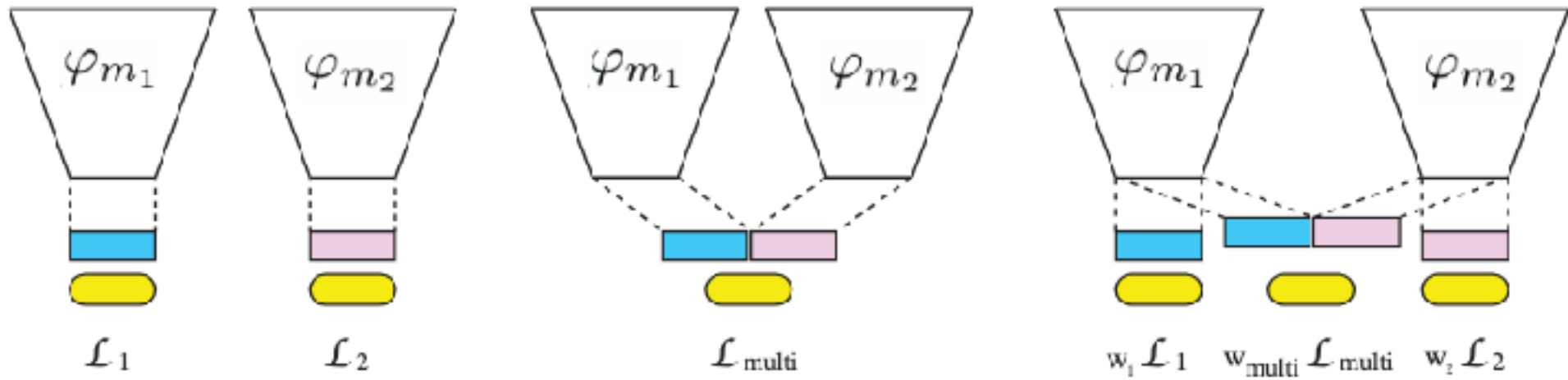


Wortwein et al., Beyond Additive Fusion:
Learning Non-Additive Multimodal
Interactions, Findings-EMNLP 2022

Overfitting



Gradient Blending can help:
Supervision=(on Unimodal) + (Multimodal)



Wang, Weiyao, Du Tran, and Matt Feiszli. "What makes training multi-modal classification networks hard?." *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020.

Recipe Card: **Supervised Cross-Attention Fusion**

Type of Learning: **Supervised multimodal prediction**

Data type: **Paired multimodal observations with target values**

Data

- 1- Load paired labeled data : $\mathcal{D} = \left\{ (x_i^A, x_i^B, y_i) \right\}_{i=1}^N$
 - Each pair is: $(x_i^A, x_i^B) \longrightarrow y_i$.
- 2- Split into train, validation, and test data:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}.$$
 - Keep dependent observations together when needed
- 3- Build mini-batches: $\mathcal{B} = \left\{ (x_i^A, x_i^B, y_i) \right\}_{i=1}^m$.

NN model

- 4- Encode each modality into a set of elements

$$z_i^A = f_A(x_i^A) \quad z_i^B = f_B(x_i^B)$$
 - Note: the architecture of f depends on each data.
- 5- Implement Cross-Attention
 - 5.1 Let modality A query modality B :

$$Q_i^A = z_i^A W_Q \quad K_i^B = z_i^B W_K, \quad V_i^B = z_i^B W_V$$
 - 5.1 Read from B and update A :

$$c_i^{A \leftarrow B} = \text{softmax}_{\text{row}} \left(\frac{Q_i^A (K_i^B)^\top}{\sqrt{d_k}} \right) V_i^B$$

$$\tilde{z}_i^A = z_i^A + c_i^{A \leftarrow B}$$
- 6- Add another network to process and predict Y :

$$\hat{y}_i^{A,B} = \tilde{f}^A(\tilde{z}_i^A).$$
 - where $\tilde{f}^A$ is the rest of the neural network after fusion.
- 7- We call the whole network: $f_{\text{multi}}(x_i^A, x_i^B) = \hat{y}_i^{A,B}$.

Training

- 8- For each mini-batch:
 - 7.1-calculate the supervised task loss: $\mathcal{L}_{\text{task}} = \frac{1}{m} \sum_{i=1}^m \ell(\hat{y}_i^{A,B}, y_i)$.
 - 7.2 Update all trainable components: $f^A, f^B, W_Q, W_K, W_V, \tilde{f}^A$.
 - 7.3 Repeat over mini-batches and epochs

Note: Use $\mathcal{D}_{\text{val}}$ for early stopping and architecture or hyperparameter choices.

Evaluation

- 9- Test if multimodality improves task, use $\mathcal{D}_{\text{test}} = \left\{ (x_i^A, x_i^B, y_i) \right\}_{i=1}^{N_{\text{test}}}$
 - 9.1-make and train two unimodal models similar to $f_{\text{multi}}(x^A, x^B)$:

$$\hat{y}_i^A = f_{\text{uni}}^A(x_i^A) \quad , \quad \hat{y}_i^B = f_{\text{uni}}^B(x_i^B)$$
 - 9.2- based on these calcite accuracies: $\text{Acc}_A, \text{Acc}_B, \text{Acc}_{\text{multi}}$
 - 9.3- If $\text{Acc}_{\text{multi}} > (\text{Acc}_A \text{ and } \text{Acc}_B)$ both multimodality improved prediction
- 10-EMAP: Test if interaction improves task, use $\mathcal{D}_{\text{test}} = \left\{ (x_i^A, x_i^B, y_i) \right\}_{i=1}^{N_{\text{test}}}$
 - 10.1- Evaluate the frozen model on every crossed pair: $s_{ij} = f_{\text{multi}}(x_i^A, x_j^B), i, j = 1, \dots, N_{\text{test}}$
 - 10.2-Compute: $\bar{f}_A(x_i^A) = \frac{1}{N_{\text{test}}} \sum_{j=1}^{N_{\text{test}}} s_{ij} \quad ; \quad \bar{f}_B(x_i^B) = \frac{1}{N_{\text{test}}} \sum_{j=1}^{N_{\text{test}}} s_{ji} \quad ; \quad \mu = \frac{1}{N_{\text{test}}^2} \sum_{p=1}^{N_{\text{test}}} \sum_{q=1}^{N_{\text{test}}} s_{pq}$
 - 10.3-EMAP is: $\tilde{f}_{\text{EMAP}}(x_i^A, x_i^B) = \bar{f}_A(x_i^A) + \bar{f}_B(x_i^B) - \mu$
 - 10.4-Then compare the original vs EMAP predictions:

$$\hat{y}_i^{A,B} = f_{\text{multi}}(x_i^A, x_i^B), \quad \hat{y}_i^{\text{EMAP}} = \tilde{f}_{\text{EMAP}}(x_i^A, x_i^B).$$
 - 10.5-And from prediction accuracy we get the amount of "interaction gain":

$$\Delta_{\text{int}} = \text{Acc}_{\text{multi}} - \text{Acc}_{\text{EMAP}}$$
 - 10.6- Now we can see if Δ_{int} is lager and positive, interaction has an effect

ingredient: **Attention Operator block (PyTorch implementation)**

Input/output

1- Input tensors

Query source: $Z \in [B, T', d_z]$

Key/Value source: $X \in [B, T, d_x]$

2- Output tensor:

Attention output: $Y \in [B, T', d_v]$

variable

1-Learnable projections: (Use nn.Linear(...)):

$W_Q : d_z \rightarrow d_k, W_K : d_x \rightarrow d_k, W_V : d_x \rightarrow d_v$

In PyTorch, where B is batch size:

$Q = W_Q(Z) \in [B, T', d_k]$

$K = W_K(X) \in [B, T, d_k]$

$V = W_V(X) \in [B, T, d_v]$

Computation

1-Attention computation:

$$E = \frac{QK^T}{\sqrt{d_k}} \in [B, T', T]$$

In PyTorch transpose Keys on the last two dims use
batched matmul:

$$A = \text{softmax}(E, \text{dim} = -1) \in [B, T', T]$$

2- $Y = AV \in [B, T', d_v]$

